Actividad 1: Levantar PostgreSQL con Docker

Introducción

Para garantizar la persistencia de los datos del sistema **Gestión de Citas Inteligente**, se configuró una base de datos **PostgreSQL** utilizando **Docker Compose**.
Esta estrategia permite levantar rápidamente los servicios necesarios del proyecto, mantener un entorno uniforme entre los miembros del equipo y facilitar la integración de los microservicios con la base de datos.

Creación del archivo docker-compose.yml

Se agregó el archivo docker-compose.yml en la raíz del monorepo del proyecto. Este archivo permite definir y ejecutar los contenedores necesarios para el sistema.

La configuración incluyó al menos el servicio de PostgreSQL, definiendo imagen, puertos, variables de entorno y volumen de persistencia.

```
version: '3.8'


services:
  postgres:
    image: postgres:16
    container_name: citas-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin1234
      POSTGRES_DB: turnosdb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

volumes:

  postgres_data:

Esta configuración permite levantar una instancia de PostgreSQL accesible en el puerto 5432.

Ejecución de Docker Compose

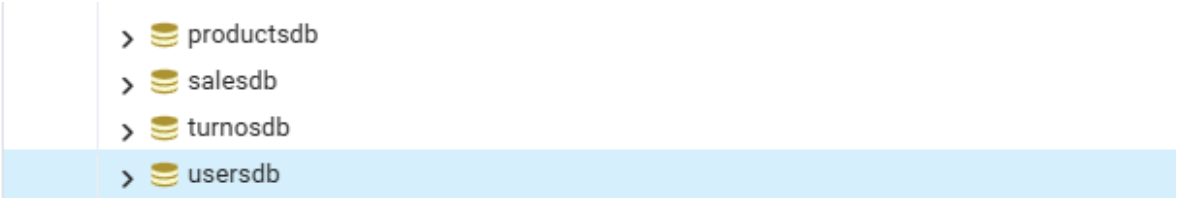
Una vez definido el archivo de configuración, se ejecutó el siguiente comando en la raíz del proyecto:

docker compose up -d

Este comando permitió iniciar los contenedores en segundo plano, dejando PostgreSQL disponible para ser consumido por el microservicio.

Verificación en pgAdmin

Después de levantar el contenedor, se accedió a **pgAdmin** para comprobar que la base de datos fue creada correctamente.
Se verificó la existencia de la base de datos turnosdb y su disponibilidad para recibir conexiones desde el microservicio.



Arranque del microservicio y creación automática de tabla

Posteriormente se arrancó el microservicio principal del proyecto, configurado con Spring Boot y JPA/Hibernate.
Gracias a la propiedad ddl-auto: update, la tabla correspondiente a la entidad principal fue creada automáticamente al iniciar la aplicación.

Se verificó que la tabla turnos fue generada correctamente dentro de la base de datos PostgreSQL.

Conclusión Actividad 1

La integración de PostgreSQL con Docker permitió contar con una base de datos funcional, portable y fácil de levantar. Además, la creación automática de tablas validó la correcta conexión entre el microservicio y la base de datos.

Actividad 2: Implementar 2 o más entidades con relación

Introducción

Con el fin de representar mejor el dominio del sistema, se implementaron al menos dos entidades relacionadas dentro del microservicio.
Esto permitió modelar adecuadamente la información y reflejar las relaciones reales del negocio dentro de la base de datos.

En el proyecto **Gestión de Citas Inteligente**, se trabajó con las entidades:

- **Usuario**
- **Turno**

La relación establecida fue:

- Un **Usuario** puede tener **muchos Turnos**
- Un **Turno** pertenece a un solo **Usuario**

Esto corresponde a una relación:

@OneToMany / @ManyToOne

Entidad Usuario

La entidad Usuario representa a las personas registradas en el sistema.
Sus atributos principales pueden incluir:

- idUsuario
- nombre
- correo
- contraseña
- rol

Entidad Turno

La entidad Turno representa una cita médica agendada dentro del sistema.
Sus atributos principales incluyen:

- idTurno
- doctor
- especialidad
- estado
- fechaCreacion
- fechaHora
- usuario

Relación entre entidades

La relación entre ambas entidades se definió de la siguiente forma:

- Usuario contiene una lista de turnos
- Turno referencia a un usuario

Esto permite consultar tanto los turnos asociados a un usuario como el usuario asociado a un turno.

Repositories implementados

Se implementaron repositorios para ambas entidades utilizando Spring Data JPA:

- UsuarioRepository
- TurnoRepository

Estos repositorios permiten acceder a la base de datos y realizar operaciones CRUD sin necesidad de escribir SQL manual para las operaciones básicas.

Services implementados

Se implementó la capa de servicios para encapsular la lógica de negocio del sistema:

- UsuarioService
- TurnoService

Esta capa permite centralizar reglas de negocio, validaciones y coordinación entre controladores y repositorios.

Controllers implementados

Se desarrollaron controladores REST para exponer endpoints para ambas entidades:

- UsuarioController
- TurnoController

Esto permitió realizar operaciones como:

- crear usuario
- listar usuarios
- crear turnos
- consultar turnos por usuario
- actualizar información
- eliminar registros

Uso de DTOs

Para mejorar la estructura de entrada y salida de datos, se implementaron **DTOs (Data Transfer Objects)**.

Estos DTOs permiten exponer únicamente la información necesaria al cliente, evitando enviar entidades completas y mejorando la seguridad y organización del sistema.

Se utilizaron DTOs tanto para requests como para responses.

Ejemplos:

- UsuarioRequestDTO
- UsuarioResponseDTO
- TurnoRequestDTO
- TurnoResponseDTO

Verificación con Postman

Se probaron los endpoints de ambas entidades utilizando Postman, verificando:

- creación de usuarios
- creación de turnos asociados a usuarios
- consulta de registros
- actualización de datos
- eliminación de registros

También se comprobó que las respuestas se devolvieran usando DTOs y no directamente las entidades.

Conclusión Actividad 2

La implementación de entidades relacionadas permitió representar mejor el dominio del sistema y estructurar la persistencia de datos de manera coherente. El uso de DTOs ayudó a mejorar la claridad y seguridad de las respuestas expuestas por la API.

Actividad 3: Manejo de Errores y Validación

Introducción

Para fortalecer la calidad del microservicio, se implementó un sistema de validación de datos y manejo global de errores.
Esto permite evitar el almacenamiento de información inválida y ofrecer respuestas claras cuando el cliente envía datos incorrectos.

Implementación de GlobalExceptionHandler

Se creó una clase GlobalExceptionHandler para capturar excepciones de forma centralizada y devolver respuestas estructuradas al cliente.

Esta clase permite manejar errores como:

- recurso no encontrado
- datos inválidos
- errores de validación
- excepciones generales del sistema

De esta manera, el cliente recibe mensajes claros y consistentes.

Validaciones en los DTOs

Se agregaron validaciones en los DTOs de request usando anotaciones como:

- @NotBlank
- @Email
- @Size

Estas validaciones se aplicaron a campos que requieren restricciones específicas.

Ejemplos de validación

- @NotBlank para evitar campos vacíos
- @Email para validar correos electrónicos
- @Size para controlar longitud mínima y máxima de cadenas

Esto ayuda a garantizar que los datos recibidos por la API cumplan con el formato esperado.

Pruebas con datos inválidos

Se realizaron pruebas enviando datos inválidos desde Postman, por ejemplo:

- nombre vacío
- correo con formato incorrecto
- contraseña demasiado corta

En estos casos, el sistema respondió con código:

400 Bad Request

y con un mensaje claro indicando el motivo del error.

Ejemplo de respuesta de error

```
{
  "mensaje": "El correo no tiene un formato válido",
  "estado": 400
}
```

o una estructura similar según la implementación realizada.

Conclusión Actividad 3

La implementación del manejo global de errores y las validaciones en DTOs permitió mejorar la robustez del sistema, evitando el ingreso de datos inválidos y brindando mensajes claros al cliente.

Actividad 4: Commit, PR y Merge

Introducción

Como parte del flujo de trabajo colaborativo del equipo, se siguieron buenas prácticas de control de versiones mediante commits descriptivos, uso de ramas de funcionalidad, Pull Requests y revisiones de código antes de integrar cambios a la rama principal de desarrollo.

Commit con convención definida

Se realizó un commit siguiendo la convención indicada por la actividad:

feat(service-one): add PostgreSQL persistence

En el contexto del proyecto, esta convención se utilizó para reflejar claramente el tipo de cambio realizado y el servicio afectado.

El uso de commits descriptivos mejora la trazabilidad del desarrollo y facilita la revisión del historial del repositorio.

Push a feature branch y creación del PR

Los cambios se subieron a una rama de funcionalidad y luego se creó un **Pull Request** hacia la rama:

develop

Esto permitió mantener aislado el desarrollo hasta que fuera revisado y aprobado.

Review cruzado

Se realizó una revisión de código por parte de otro integrante del equipo o de otro equipo, validando la implementación realizada.

Este proceso permitió verificar:

- estructura correcta del código
- funcionamiento del microservicio
- integración con PostgreSQL
- cumplimiento de buenas prácticas

Merge aprobado y verificación en develop

Una vez aprobado el Pull Request, se realizó el merge hacia la rama develop. Posteriormente se verificó que el proyecto seguía funcionando correctamente con la base de datos levantada en Docker.

Esto confirmó la correcta integración de la persistencia al entorno principal de desarrollo.

Conclusión Actividad 4

El uso de commits descriptivos, Pull Requests y revisiones de código permitió integrar los cambios al proyecto de forma controlada, segura y colaborativa.

Preparación para el MVP (Semana 5)

Estado del proyecto para la Semana 5

De acuerdo con los requerimientos de preparación para el MVP, el proyecto cuenta con los siguientes avances:

1. Al menos un microservicio con CRUD completo y PostgreSQL

Se desarrolló el microservicio principal con operaciones CRUD completas y persistencia en PostgreSQL, validando su funcionamiento mediante Postman y pgAdmin.

2. API Gateway apuntando al microservicio

El sistema cuenta con un API Gateway configurado para enrutar solicitudes al microservicio correspondiente, centralizando el acceso desde el cliente.

3. Docker Compose levantando los servicios necesarios

Se configuró Docker Compose para levantar PostgreSQL y los servicios requeridos por el proyecto.

4. README.md actualizado

El repositorio incluye un archivo README.md actualizado con la información del proyecto y las instrucciones básicas de ejecución.

5. Evidencia en GitHub

Se cuenta con evidencia del trabajo realizado en GitHub, incluyendo:

- commits descriptivos
- Pull Requests
- Issues cerrados
- tareas en la columna Done del tablero del proyecto

Conclusión General

Las actividades desarrolladas permitieron consolidar la base técnica del MVP del proyecto **Gestión de Citas Inteligente**.
Se logró integrar persistencia con PostgreSQL, modelar entidades relacionadas, validar datos, manejar errores, trabajar con buenas prácticas de Git y dejar evidencia clara del avance del sistema en GitHub.